

Lecture 6: Data Acquisition - Scraping Static and Dynamic Websites

James Sears*

AFRE 891/991 SS 25

Michigan State University

*Parts of these slides are adapted from [“Data Science for Economists”](#) by Grant McDermott.

Table of Contents

1. Prologue
2. Scraping Static Websites
3. Interacting with Static Websites
4. Scraping Dynamic Websites

Prologue

Packages for Today

Today we'll pick up with scraping static websites.

For this we'll need the following packages:

Static:

```
pacman::p_load(lubridate, rvest, stringr, tidyverse, tidyr)
```

and for dynamic scraping:

```
pacman::p_load(chromote, selenider)
```

Scraping Static Websites

Scraping Static Sites

Last time we saw how to use **rvest** functions to scrape information from **HTML Tables**.

- **Biggest Challenge:** finding the right CSS selectors¹.

Let's work through an example navigating another site's structure to get familiar with more helpful functions.

¹. For more on all the ways to construct CSS selectors, check out [this handy W3 Guide](#)

Example 2: Des Linden

Let's say you got excited by all this marathon talk and wanted to gather more data about one of the most storied American marathoners:\ **Des Linden**

- A native Michigander (originally from Charlevoix, MI)!



Example 2: Des Linden

You know she ran the 2024 Boston Marathon, so you decide to scrape Des' [2024 Boston Marathon results page](#):



128TH BOSTON
MARATHON[®]



BANK OF AMERICA

Results

Leaderboard

Tracking

My Runners

Person Details

Participant

Name	Linden, Desiree
Team	—
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	16
Place (g)	3
Place (Total)	95
Time Total (net)	02:28:27
Time Total (gun)	02:28:27
min/mile	05:40

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:04:04AM	00:17:03	17:03	05:30	10.94
10K	10:21:21AM	00:34:20	17:17	05:34	10.79
15K	10:38:42AM	00:51:41	17:21	05:35	10.74
20K	10:55:52AM	01:08:51	17:10	05:32	10.86
HALF	10:59:42AM	01:12:41	03:50	05:38	10.66
25K	11:13:11AM	01:26:10	13:29	05:34	10.80
30K	11:31:08AM	01:44:07	17:57	05:47	10.39
20 Miles	11:39:06AM	01:52:05	07:58	05:52	10.23
21 Miles	11:45:18AM	01:58:17	06:12	06:12	9.68
35K	11:49:31AM	02:02:30	04:13	05:39	10.64
23 Miles	11:56:54AM	02:09:53	07:23	05:56	10.14
	12:02:35PM	02:15:34	05:41	05:41	10.56
40K	12:07:33PM	02:20:32	04:58	05:47	10.38
25.2 Miles	12:09:45PM	02:22:44	02:12	06:23	9.45
Finish Net	12:15:28PM	02:28:27	05:43	05:37	10.70

Features



Marathonphoto.com

Results Adjustment Request

Example 2: Boston 2024 Results

To start, read in the page's HTML.

```
des_24 ← read_html("https://results.baa.org/2024/?content=detail&fpid=li:
```

Example 2: Boston 2024 Results

What do we see when we look at the page?

```
des_24
```

```
## {html_document}  
## <html>  
## [1] <head>\n<title>Boston Athletic Association</title>\n<meta http-equiv="Con ...  
## [2] <body class="pid-list content-detail view-public branch-production site-2 ...
```

1. The page's head information (`<head>`)
 - Contains metadata about the page (not actually displayed)
2. The page's body contents (`<body>`)
 - This is where all the stuff is

Example 2: Boston 2024 Results

Look at the body's **child elements** with `html_children()`

- all 10 elements nested under `body`

```
html_element(des_24, "body") %>% html_children()

## {xml_nodeset (10)}
## [1] <header class="hidden-xs"><div class="container">\n<div class="logo row
## [2] <nav class="navbar navbar-default navbar-top navbar-fixed-top"><div cla
## [3] <script async src="https://www.googletagmanager.com/gtag/js"></script>
## [4] <script>\r\n          \r\n          var mt_tracker = {}; \r\n          \r\n
## [5] <section class="section-nav container"><nav class="navbar navbar-defaul
## [6] <section class="section-main"><div class="container" id="main-content">
## [7] <footer><div class="container">\n<div class="row">\n<div class="footer-
## [8] <div class="popup-info" style="display:none;" id="popup-info"></div>
## [9] <div class="popup" style="display:none;" id="popup"></div>
## [10] <div id="cconsent-overlay" style="display:none;"></div>
```

Example 2: Boston 2024 Results

Look at the body's **child elements** with `html_children()`

- all 10 elements nested under `body`

This matches the **page source** (access with "Inspect")

```
<body class="pid-list content-detail view-public branch-production site-2024_boston event  
-R"> == $0  
  <header class="hidden-xs">...</header>  
  <nav class="navbar navbar-default navbar-top navbar-fixed-top">...</nav>  
  <!-- header end -->  
  <!-- main start -->  
  <script async src="https://www.googletagmanager.com/gtag/js"></script>  
  <script>...</script>  
  <section class="section-nav container">...</section>  
  <section class="section-main">...</section>  
  <!-- section-main -->  
  <!-- main end -->  
  <!-- footer start -->  
  <footer>...</footer>  
  <!-- footer section end -->  
  <div class="popup-info" style="display:none;" id="popup-info"></div>  
  <div class="popup" style="display:none;" id="popup"></div>  
  <div id="cconsent-overlay" style="display:none;"></div>
```

Example 2: Boston 2024 Results

Let's start by grabbing all the information from the left half of the page.

Using SelectorGadget, identify the selector for the left-side contents.

Person Details

Participant

Name	Linden, Desiree
Team	
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	16
Place (g)	3
Place (Total)	95
Time Total (net)	02:28:27
Time Total (gun)	02:28:27
min/mile	05:40

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:04:04AM	00:17:03	17:03	05:30	10.94
10K	10:21:21AM	00:34:20	17:17	05:34	10.79
15K	10:38:42AM	00:51:41	17:21	05:35	10.74
20K	10:55:52AM	01:08:51	17:10	05:32	10.86
HALF	10:59:42AM	01:12:41	03:50	05:38	10.66
25K	11:13:11AM	01:26:10	13:29	05:34	10.80
30K	11:31:08AM	01:44:07	17:57	05:47	10.39
20 Miles	11:39:06AM	01:52:05	07:58	05:52	10.23
21 Miles	11:45:18AM	01:58:17	06:12	06:12	9.68
35K	11:49:31AM	02:02:30	04:13	05:39	10.64
23 Miles	11:56:54AM	02:09:53	07:23	05:56	10.14
24 Miles	12:02:35PM	02:15:34	05:41	05:41	10.56
40K	12:07:33PM	02:20:32	04:58	05:47	10.38
25.2 Miles	12:09:45PM	02:22:44	02:12	06:23	9.45
Finish Net	12:15:28PM	02:28:27	05:43	05:37	10.70

Example 2: Boston 2024 Results

```
des_24_l ← html_element(des_24, ".channel-left")
```

What's here?

```
des_24_l
```

```
## {html_node}  
## <div class="col col-xs-12 col-md-6 detail-channel channel-left">  
## [1] <div class="detail-box box-general">\n<h3>Participant\n</h3>\n<table cla  
## [2] <div class="detail-box box-totals">\n<h3>Result\n</h3>\n<table class="ta
```

Two elements:

1. detail-box box-general: class of top half div
2. detail-box box-totals: class of bottom half div

Example 2: Boston 2024 Results

Start by grabbing the top half info with the full class:

```
des_24_lu ← html_element(des_24_l, ".detail-box.box-general")
```

Q: did we need the first half (`detail-box`)?

```
identical(des_24_lu,  
          html_element(des_24_l, ".box-general")  
          )
```

```
## [1] TRUE
```

A: No! since there's only one `box-general` here we can refer to it directly.

Elements vs. Element

We can see the difference with `html_element()` and `html_elements()`

- `html_element()` retrieves the **HTML nodes** directly
 - same length as the input
 - If a single node, just one thing gets returned
 - if a nodeset, returns a same-length vector
- `html_elements()` retrieves the **nodeset itself**
 - Contains all nodes that match

```
html_element(des_24, ".detail-box")
```

```
## {html_node}
```

```
## <div class="detail-box box-general">
```

```
## [1] <h3>Participant\n</h3>
```

```
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-__fullname">
```

```
html_elements(des_24, ".detail-box")
```

```
## {xml_nodeset (5)}
```

```
## [1] <div class="detail-box box-general">
```

```
## [2] <div class="detail-box box-total">
```

```
## [3] <div class="detail-box box-state">
```

```
## [4] <div class="detail-box box-split">
```

```
## [5] <div class="detail-box box-featu">
```

Example 2: Boston 2024 Results

What does the left upper box contain?

```
des_24_lu
```

```
## {html_node}  
## <div class="detail-box box-general">  
## [1] <h3>Participant\n</h3>  
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-__fullname">
```

What we grabbed *itself* contains two elements:

1. An `h3` title, and
2. a table (`table`)

Example 2: Boston 2024 Results

Alternatively, to view **all elements AND all child elements**, use the wildcard selector "*" in `html_elements()`

- Ignores structure/nesting

```
html_elements(des_24_lu, "*")
```

```
## {xml_nodeset (24)}
## [1] <h3>Participant</h3>
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-__fullname"> ...
## [3] <tbody>\n<tr class=" f-__fullname">\n<th class="desc">Name</th>\n<td cla ...
## [4] <tr class=" f-__fullname">\n<th class="desc">Name</th>\n<td class="f-__f ...
## [5] <th class="desc">Name</th>
## [6] <td class="f-__fullname last">Linden, Desiree</td>
## [7] <tr class="list-highlight f-team">\n<th class="desc">Team</th>\n<td clas ...
## [8] <th class="desc">Team</th>
## [9] <td class="f-team last"><span class="text-muted">–</span></td>
## [10] <span class="text-muted">–</span>
## [11] <tr class=" f-start_no_text">\n<th class="desc">Bib Number</th>\n<td cla ...
## [12] <th class="desc">Bib Number</th>
## [13] <td class="f-start_no_text last">118</td>
## [14] <tr class="list-highlight f- type age class">\n<th class="desc">Division
```

Grabbing Text

If we want to grab the text from the third-level heading `h3`, we can use

1. `html_text()`: retrieves the **raw, underlying text**
2. `html_text2()`: retrieves the text **as displayed in a browser**

```
html_elements(des_24_lu, "h3") %>% html_elements(des_24_lu, "h3") %>%  
  html_text()                                html_text2()
```

```
## [1] "Participant\n"
```

```
## [1] "Participant"
```

Here the difference is whether we keep the line break special character `\n` or not.

Example 2: Boston 2024 Results

Grabb the (full) table with `html_table()`

- `html_table()` recognizes the tabular format of an HTML table, automatically converts to a tibble data frame (easiest option)

```
part_tab <- des_24_lu %>%  
  html_element("table.table-condensed") %>%  
  html_table()
```

```
part_tab
```

```
## # A tibble: 6 × 2  
##   X1          X2  
##   <chr>      <chr>  
## 1 Name      "Linden, Desiree"  
## 2 Team      "-"  
## 3 Bib Number "118"  
## 4 Division  "40-44"  
## 5 State/ Province "MI"  
## 6 My Runner  ""
```

Example 2: Boston 2024 Results

Challenge: get the contents of the "Result" table (bottom-left)

Workflow:

- Start with `des_24_1`
- Use `html_element()` (likely twice) to get the bottom table
- Format as a data frame with `html_table()`

128TH BOSTON MARATHON | BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Person Details

Participant

Name	Linden, Desiree
Team	—
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	+

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:04:04AM	00:17:03	17:03	05:30	10.94
10K	10:21:21AM	00:34:20	17:17	05:34	10.79
15K	10:38:42AM	00:51:41	17:21	05:35	10.74
20K	10:55:52AM	01:08:51	17:10	05:32	10.86
HALF	10:59:42AM	01:12:41	03:50	05:38	10.66
25K	11:13:11AM	01:26:10	13:29	05:34	10.80
30K	11:31:08AM	01:44:07	17:57	05:47	10.39
20 Miles	11:39:06AM	01:52:05	07:58	05:52	10.23
21 Miles	11:45:18AM	01:58:17	06:12	06:12	9.68
35K	11:49:31AM	02:02:30	04:13	05:39	10.64
23 Miles	11:56:54AM	02:09:53	07:23	05:56	10.14
24 Miles	12:02:35PM	02:15:34	05:41	05:41	10.56
40K	12:07:33PM	02:20:32	04:58	05:47	10.38
25.2 Miles	12:09:45PM	02:22:44	02:12	06:23	9.45
Finish Net	12:15:28PM	02:28:27	05:43	05:37	10.70

Result

Place	16
Place (g)	3
Place (Total)	95
Time Total (net)	02:28:27
Time Total (gun)	02:28:27
min/mile	05:40

Features

MarathonPhoto.com

Results Adjustment Request

Example 2: Boston 2024 Results

```
res_tab <- des_24_l %>%  
  html_element(".box-totals") %>%  
  html_element(".table-condensed") %>%  
  html_table()  
head(res_tab)
```

```
## # A tibble: 6 × 2  
##   X1                X2  
##   <chr>            <chr>  
## 1 Place           16  
## 2 Place (g)       3  
## 3 Place (Total)   95  
## 4 Time Total (net) 02:28:27  
## 5 Time Total (gun) 02:28:27  
## 6 min/mile        05:40
```

Example 2: Boston 2024 Results

Now that we have the left side stuff, let's grab the right side!

Conveniently, the selector follows the same naming convention

```
des_24_r ← des_24 %>% html_element(".channel-right")
des_24_r

## {html_node}
## <div class="col col-xs-12 col-md-6 detail-channel channel-right">
## [1] <div class="detail-box box-state">\n<h3>Race Status\n</h3>\n<table class
## [2] <div class="detail-box box-splits">\n<h3>Splits\n</h3>\n<div class="tabl
## [3] <div class="detail-box box-features">\n<h3>Features\n</h3>\n<table class
```

Example 2: Boston 2024 Results

Here we can see that all the right-side info is *also* being stored in HTML tables.

We *could* manually grab both the tables again... or we can take advantage of `html_elements()` retrieving all matches at once

- Pull *all* `table` elements from `des_24`

```
html_elements(des_24, ".table")
```

```
## {xml_nodeset (5)}  
## [1] <table class="table table-condensed"><tbody>\n<tr class=" f-__fullname">  
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-place_all">\n<td colspan="2">Place  
## [3] <table class="table table-condensed"><tbody>\n<tr class=" f-race_status">  
## [4] <table class="table table-condensed table-striped">\n<thead><tr>\n<th class=" f-race_status">Race  
## [5] <table class="table table-condensed"><tbody><tr>\n<td class=" last"><div>
```

```
splits <- html_elements(des_24, ".table")[4] %>% html_table()
```

Example 2: Boston 2024 Results

Finally, let's get the url links from the "Features" box buttons

**128TH BOSTON MARATHON[®]**BANK OF AMERICA

[Results](#) [Leaderboard](#) [Tracking](#) [My Runners](#)

Person Details

Participant

Name	Linden, Desiree
Team	—
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	16
Place (g)	3
Place (Total)	95
Time Total (net)	02:28:27
Time Total (gun)	02:28:27
min/mile	05:40

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:04:04AM	00:17:03	17:03	05:30	10.94
10K	10:21:21AM	00:34:20	17:17	05:34	10.79
15K	10:38:42AM	00:51:41	17:21	05:35	10.74
20K	10:55:52AM	01:08:51	17:10	05:32	10.86
HALF	10:59:42AM	01:12:41	03:50	05:38	10.66
25K	11:13:11AM	01:26:10	13:29	05:34	10.80
30K	11:31:08AM	01:44:07	17:57	05:47	10.39
20 Miles	11:39:06AM	01:52:05	07:58	05:52	10.23
21 Miles	11:45:18AM	01:58:17	06:12	06:12	9.68
35K	11:49:31AM	02:02:30	04:13	05:39	10.64
23 Miles	11:56:54AM	02:09:53	07:23	05:56	10.14
24 Miles	12:02:35PM	02:15:34	05:41	05:41	10.56
40K	12:07:33PM	02:20:32	04:58	05:47	10.38
25.2 Miles	12:09:45PM	02:22:44	02:12	06:23	9.45
Finish Net	12:15:28PM	02:28:27	05:43	05:37	10.70

Features

[Results Adjustment Request](#)

[Marathonphoto.com](#)

Retrieving Links

Workflow:

- Start with `des_24`

```
des_24
```

```
## {html_document}  
## <html>  
## [1] <head>\n<title>Boston Athletic A.  
## [2] <body class="pid-list content-de
```

Retrieving Links

Workflow:

- Start with `des_24`
- Navigate to the right side
(`.channel-right`)

```
des_24 %>%
```

```
  html_element(".channel-right")
```

```
## {html_node}
```

```
## <div class="col col-xs-12 col-md-6 d
```

```
## [1] <div class="detail-box box-state
```

```
## [2] <div class="detail-box box-split
```

```
## [3] <div class="detail-box box-featu
```

Retrieving Links

Workflow:

- Start with `des_24`
- Navigate to the right side
(`.channel-right`)
- Navigate into the feature box
(`.box-features`)

```
des_24 %>%  
  html_element(".channel-right") %  
  html_element(".box-features")  
  
## {html_node}  
## <div class="detail-box box-features"  
## [1] <h3>Features\n</h3>  
## [2] <table class="table table-condens
```

Retrieving Links

Workflow:

- Start with `des_24`
- Navigate to the right side
(`.channel-right`)
- Navigate into the feature box
(`.box-features`)
- Retrieve the elements matching
the button class (`a.btn.btn-
primary.btn-block`)

```
des_24 %>%  
  html_element(".channel-right") %  
  html_element(".box-features") %>  
  html_elements("a.btn.btn-primary  
## {xml_nodeset (2)}  
## [1] <a class="btn btn-primary btn-bl  
## [2] <a class="btn btn-primary btn-bl
```

Retrieving Links

Workflow:

- Start with `des_24`
- Navigate to the right side (`.channel-right`)
- Navigate into the feature box (`.box-features`)
- Retrieve the elements matching the button class (`a.btn.btn-primary.btn-block`)
- Use `html_attr("href")` to retrieve all values of the `href` attributes

```
des_24 %>%  
  html_element(".channel-right") %  
  html_element(".box-features") %>  
  html_elements("a.btn.btn-primary  
html_attr("href")
```

```
## [1] "https://www.marathonfoto.com/In  
## [2] "https://www.baa.org/boston-mara
```

Structured v Unstructured

Here nearly everything was nicely formatted in HTML tables, which `html_table()` automatically converts into a structured format.

If what you want is unstructured/not together, you'll need to figure out the **individual selectors** and put them together in a **custom data object**

```
data.frame(  
  athlete = "Des Linden",  
  title = html_element(des_24, "a.navbar-brand") %>% html_element(".hidden"),  
  back_link = html_element(des_24, ".tt-help") %>% html_attr("href")  
)
```

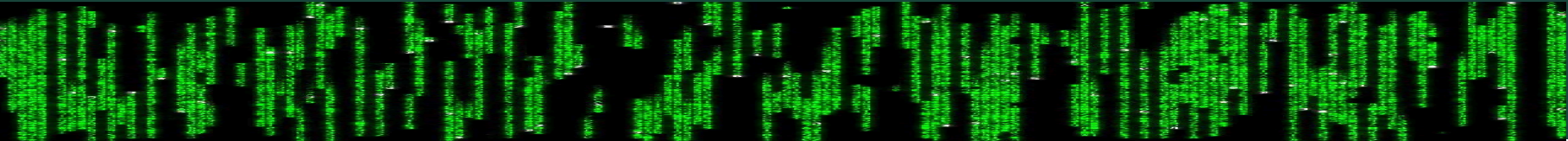
```
##           athlete                                title  
## 1 Des Linden Boston Athletic Association | 2024  
##  
## 1 /2024/?event=R&event_main_group=runner&pid=start&search%5Bsex%5D=W&search%
```

Summary: Static Scraping

When scraping static web content rendered server-side:

- Start by finding the relevant CSS selectors
- Use the **rvest** package to read in the HTML document and parse it
 - Tabular workflow: `read_html(URL) %>%
html_elements(CSS_Selectors) %>% html_table()`
 - Might need other functions depending on content type (e.g.
`html_text/text2(), html_attr(), html_children()`)

Interacting with Static Sites



While scraping a **known url** is pretty cool, we can do more than that.

We already saw one way to progress through a url chain, but we can **go deeper**.

We can go **into.the.matrix**.

Interacting with Static Sites

Let's go back to the beginning with our Boston Marathon 2024 workflow.

Suppose rather than just Des' results, you wanted to

1. Navigate back to the main results search page
 1. Enter custom search term (i.e. specific age group/gender) and execute a search from the home page
 2. Retrieve all results from the first results page
 3. Navigate to the next page and repeat the process

128TH BOSTON MARATHON

BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

Search

Group: Runner

Sub Group: Runner

Last Name:

First Name:

Bib Number:

Search

Reset

Advanced Search

Result Lists

Group: Runner

Sub Group: Runner

Gender: Men

Age Group: All

Results/Page: 25

Show Results

Interacting with Web Forms

To **interact with web forms**, we'll use the below workflow:

- Retrieve a target form with `html_form()`
- Fill in the form's required elements with `html_form_set()`
- Submit the form with `html_form_submit()`
- Read the resulting url with `html_read()`.

Let's practice this by trying to retrieve the top 50 results for my age group (Men 18-39)

Interacting with Web Forms

Start by navigating to the [Boston Marathon Results Search](#) (the main search page).

128TH BOSTON MARATHON | BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

▼ Search

Group Runner ▼

Sub Group Runner ▼

Last Name

First Name

Bib Number

Search **Reset**

► Advanced Search

Result Lists

Group Runner ▼

Sub Group Runner ▼

Gender Men ▼

Age Group All ▼

Results/Page 25 ▼

Show Results

Interacting with Web Forms

Here, there are **multiple forms** with the same class (`form-horizontal`) but different names

- `form_search_simple`: the top-left "Search"
- `form_search_advanced`: the bottom-left "Advanced Search"
- `form_lists_default`: the top-right "Results List" search

```
bos_srch ← read_html("https://results.baa.org/2024/?pid=start&pidp=start")
html_form(bos_srch)

## [[1]]
## <form> 'form_search_simple' (POST https://results.baa.org/2024/?pid=search&pidp=search)
##   <field> (hidden) lang: EN_CAP
##   <field> (hidden) startpage: start_responsive
##   <field> (hidden) startpage_type: search
##   <field> (select) event_main_group: runner
##   <field> (select) event:
##   <field> (text) search[name]:
##   <field> (text) search[firstname]:
```

Interacting with Web Forms

Use `html_form()` to extract the form we want (Results List):

To do this, we could use the `class[attr = \"value\"]` selector syntax to retrieve it as an element by name using our previous workflow

```
res_form ← html_elements(bos_srch, "form[name = \"form_lists_default\"]")
  html_form()
class(res_form)
```

```
## [1] "list"
```

But that nests the form within a list.

Interacting with Web Forms

Use `html_form()` to extract the form we want (Results List):

Alternatively, just run `html_form()` on the main page and select the list item we want using list indexing

```
html_form(bos_srch)[[3]]
```

```
## <form> 'form_lists_default' (POST https://results.baa.org/2024/?pid=list&pic
##   <field> (hidden) lang: EN_CAP
##   <field> (hidden) startpage: start_responsive
##   <field> (hidden) startpage_type: lists
##   <field> (select) event_main_group: runner
##   <field> (select) event:
##   <field> (text) search[sex]:
##   <field> (text) search[age_class]:
##   <field> (select) num_results: 25
##   <field> (button) submit:
```

Interacting with Web Forms

Every web form has several different **fields** we'll need to fill or interact with

Here we've got:

- 3 **hidden** fields that **we can't interact with**
- 3 **select** fields that have **dropdown menus**
- 2 **text subfields**: "search[sex]" and "search[age_class]" in the `event` dropdown
- 1 **button** named "submit" we'd click when manually searching

Filling Web Forms with *html_form_set()*

Let's set up our search by setting

- Age Group to 19-39
- 50 Results/Page

```
search_set ← html_form_set(res_form[[1]],  
                           "event" = "runner",  
                           "search[sex]" = "M",  
                           "search[age_class]" = "18-39",  
                           "num_results" = 50)
```

Submitting Web Forms

Submitting the form and getting the response:

```
resp ← html_form_submit(search_set) # submit the form
resp

## Response [https://results.baa.org/2024/?pid=list&pidp=start]
##   Date: 2025-02-25 02:39
##   Status: 200
##   Content-Type: text/html; charset=UTF-8
##   Size: 19.4 kB
## <!DOCTYPE html>
## <html>
## <head>
## <title>Boston Athletic Association</title>
## <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
## <meta name="apple-mobile-web-app-status-bar-style" content="black" />
## <link rel="apple-touch-icon" href="//results-static.mikatiming.com/2024/bost
## <meta name="viewport" content="width=device-width, initial-scale=1">
## <link rel="stylesheet" href="//results-static.mikatiming.com/stages/blue/lib
## <link rel="stylesheet" href="//results-static.mikatiming.com/stages/blue/lib
```

Web Form Response

Submitting the form gets us a `response` class object, containing some useful components, including the resulting url:

```
resp$url
```

```
## [1] "https://results.baa.org/2024/?pid=list&pidp=start"
```

And we can read the response url as we usually do and navigate to the results element we want:

```
# get the result url's html  
res ← read_html(resp) %>% html_elements("div.col-sm-12.row-xs") %>% html_
```

Web Form Response

And looking at our results:

```
res[2] %>% html_text()
```

```
## [1] "\n\nThere are currently no results available.\n\n"
```

Ahh dangit. Turns out that this page is **client-side** rendered

- The page's content is **dynamically-generated** based on our search.

In order to scrape a client-side site, we're going to need more tools...

Scraping Dynamic Websites

Dynamic Websites

Dynamic client-side websites are those are rendered dynamically and require **interaction** in order to obtain desired information.

While **rvest** can help us retrieve urls and element contents, it can't interact with **dynamically-generated or interactive** websites.¹

To do that, we're going to take advantage of **browser automation** methods.

¹ **rvest** has some functionality for live site interaction, but it is not as complete as the other options we'll see shortly.

Browser Automation

The main idea behind browser automation is this:

If a website requires pointing/clicking/typing to reveal the contents we want, why not have software "drive" a browser for us?

Old Approach: RSelenium

- R bindings for the Selenium webdriver
- Requires some additional software (i.e. Docker)
- Additional steps to get it working with Mac OS X

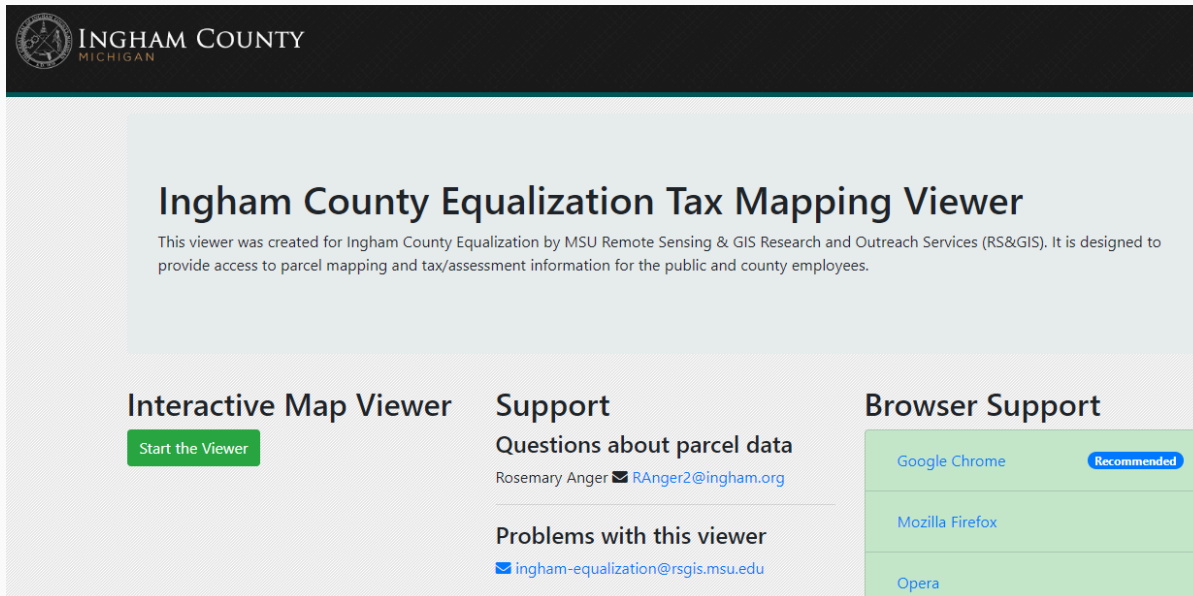
New Approach: selenider

- Uses **chromote** or Selenium automation tools to drive a Chrome browser
- Syntax more closely matches tidyverse
- Can't do some more complex things yet (i.e. click-and-hold, click a point)

Application: EL Properties

Let's work through an example of an interactive workflow for **property characteristics in East Lansing**.

Suppose you want to obtain property information for a known address within East Lansing. The city has a handy **interactive map viewer** for tax assessment info.



The screenshot shows the homepage of the Ingham County Equalization Tax Mapping Viewer. At the top is the Ingham County Michigan logo. The main heading is "Ingham County Equalization Tax Mapping Viewer". Below this is a descriptive paragraph: "This viewer was created for Ingham County Equalization by MSU Remote Sensing & GIS Research and Outreach Services (RS&GIS). It is designed to provide access to parcel mapping and tax/assessment information for the public and county employees." The page is divided into three columns. The left column, titled "Interactive Map Viewer", contains a green button labeled "Start the Viewer". The middle column, titled "Support", includes the text "Questions about parcel data" with contact information for Rosemary Anger (RAnger2@ingham.org) and "Problems with this viewer" with contact information for ingham-equalization@rsgis.msu.edu. The right column, titled "Browser Support", lists three browser options: Google Chrome (marked as "Recommended"), Mozilla Firefox, and Opera.

INGHAM COUNTY
MICHIGAN

Ingham County Equalization Tax Mapping Viewer

This viewer was created for Ingham County Equalization by MSU Remote Sensing & GIS Research and Outreach Services (RS&GIS). It is designed to provide access to parcel mapping and tax/assessment information for the public and county employees.

Interactive Map Viewer

[Start the Viewer](#)

Support

Questions about parcel data
Rosemary Anger ✉ RAnger2@ingham.org

Problems with this viewer
✉ ingham-equalization@rsgis.msu.edu

Browser Support

Google Chrome	Recommended
Mozilla Firefox	
Opera	

Application: EL Properties

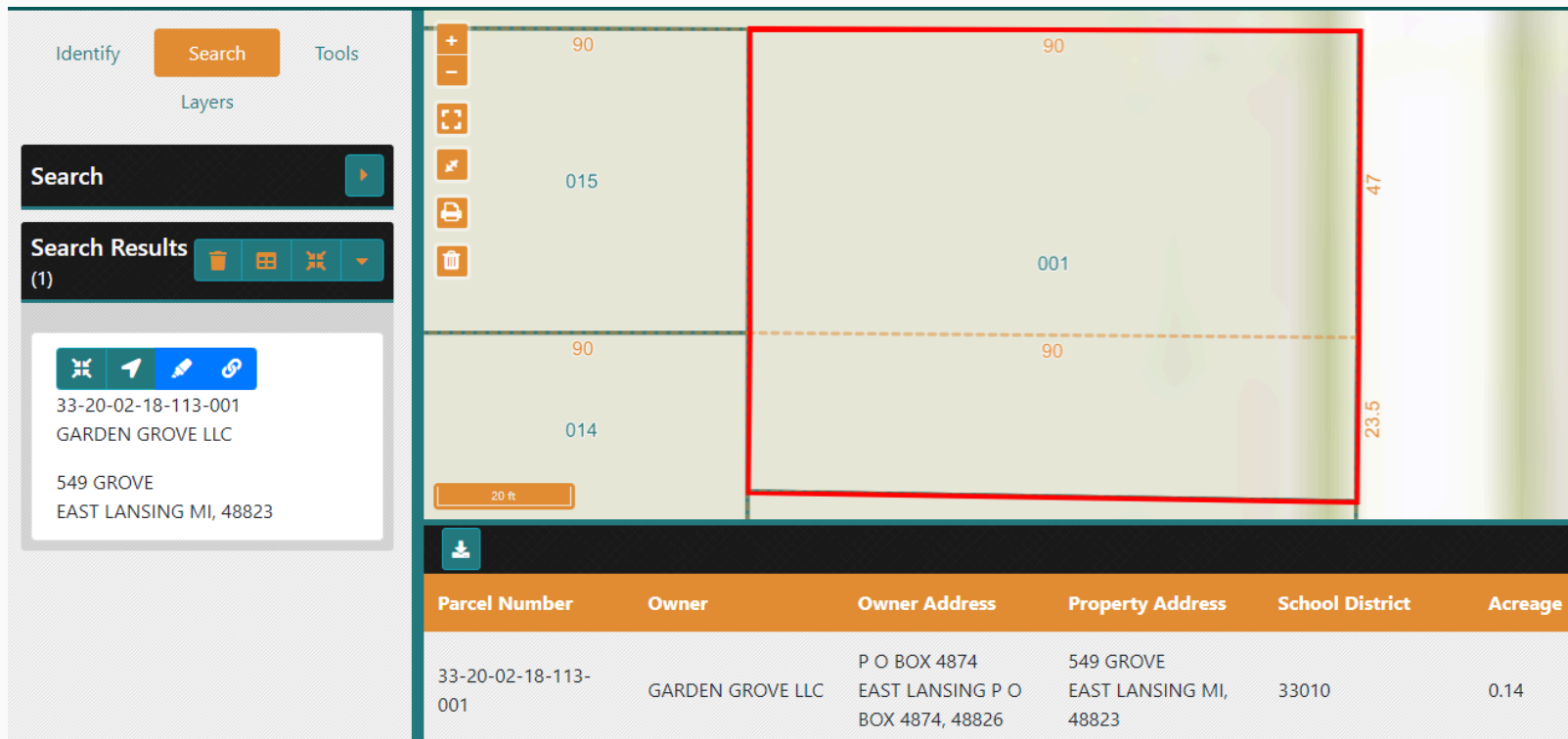
Suppose we wanted to get information about the property at 549 Grove St. from the parcel map.

Doing this takes a couple steps:

1. Launch the [Map Viewer](#)
2. Click on "Search"
3. Type address components (number and street name) into the search bar, hit Go
4. Click on "Snap to Result" button to zoom in on the parcel
5. Click on "Toggle Table" to show information

Application: EL Properties

Unfortunately, this doesn't have the property details we are interested in.



The screenshot displays a web application interface for property information. On the left, a sidebar contains a search bar, a search button, and a search results section. The search results section shows one result for the parcel number 33-20-02-18-113-001, owned by GARDEN GROVE LLC, located at 549 GROVE, EAST LANSING MI, 48823. The main area shows a map of the property with a red outline. The map includes a scale bar indicating 20 ft. The property is divided into sections with labels: 90, 015, 001, 90, 014, 47, and 23.5. Below the map is a table with property details.

Parcel Number	Owner	Owner Address	Property Address	School District	Acreage
33-20-02-18-113-001	GARDEN GROVE LLC	P O BOX 4874 EAST LANSING P O BOX 4874, 48826	549 GROVE EAST LANSING MI, 48823	33010	0.14

Application: EL Properties

Fortunately, there is a link to more information on a different site:



Application: EL Properties

Which *unfortunately* requires verification

[BS&A Online](#) [Home](#) [Services](#) [Municipalities](#) [Currently not signed in](#)

Ingham County | MI

powered by **BS&A** SOFTWARE

Search: [All Records](#) By: [Address](#)

☐ Use Advanced Address Search

SERVICES

- Public Records Search
 - [All Record Search](#)
 - [Assessing Search](#)
 - [Current Tax Search](#)
 - [Delinquent Tax Search](#)

PERFORMANCE DATA

- [MI Community Financial Dashboard](#)
- [Performance Dashboard](#)

MUNICIPALITIES

- [Select a Municipality](#)

ACCOUNT

- [Sign In](#)
- [Register](#)
- [Why Register?](#)



Security Verification

For security purposes, click the checkbox below.

Verify Code

Check the box below to verify you are a human. This security feature helps prevent automated programs from accessing unauthorized information.

☐ I'm not a robot 

reCAPTCHA
[Privacy](#) - [Terms](#)

****Disclaimer:** BS&A Software provides BS&A Online as a way for municipalities to display information online and is not responsible for the content or accuracy of the data herein. This data is provided for reference only and WITHOUT WARRANTY of any kind, expressed or inferred. Please contact your local municipality if you believe there are errors in the data.

Application: EL Properties

Which *fortunately* has the characteristics we want once we get past this!

☐ Building Information - 1676 sq ft 1 3/4 STORY (Residential)

[Back To Top](#)

General

Floor Area	1,676 sq ft
Garage Area	216 sq ft
Foundation Size	1,024 sq ft
Year Built	1922
Occupancy	Single Family
Effective Age	52 yrs
Percent Complete	0%
AC w/Separate Ducts	No
Basement Rooms	3
1st Floor Rooms	7
2nd Floor Rooms	2
Bedrooms	7

Estimated TCV	\$193,781
Basement Area	1,006 sq ft
Year Remodeled	No Data to Display
Class	C
Tri-Level	No
Heat	Forced Air w/ Ducts
Wood Stove Add-on	No
Water	Public Water
Sewer	Public Sewer
Style	1 3/4 STORY

Area Detail - Basic Building Areas

Height	Foundation	Exterior	Area	Heated
1.75 Story	Basement	Siding	870 sq ft	1.75 Story
1 Story	Basement	Brick	136 sq ft	1 Story
1 Story	Crawl Space	Siding	18 sq ft	1 Story

Workflow

Let's now work through these steps in the context of an automated browser workflow:

1. Launch an automated browser session
2. Navigate to the Map Viewer
3. Search by street name + house number
4. Retrieve displayed info
5. Click detail link
6. Pass verification test
7. Scrape desired info

1. Launch Automated Session

First, we need to **initiate an automated browser session** with

```
selenider_session().2
```

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  session = "chromote", # set backend to chromote  
  timeout = 10 # set timeout to 10 sec  
)
```

We've now created a **local session**

- Accessed anywhere in a given script/RMarkdown file
- Closes automatically when the script/file finishes
- If started within a function, closes when the function is done running

² The full list of **selenider** functions can be found [here](#)

View the Session

If you want to **view the browser session**, we need to add the argument

```
options = chromote_options(headless = F)
```

Which will open the session in your Chrome browser

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  options = chromote_options(headless = F), # view the session  
  "chromote", # set backend to chromote  
  timeout = 10 # set timeout to 10 sec  
)
```

2. Navigate to Map Viewer

Let's start by navigating to the main [Tax Mapping Viewer page](#) and launch the viewer

- `open_url("URL")`

```
open_url("https://ingham-equalization.rsgis.msu.edu/")
```

2. Navigate to Map Viewer

Check that we ended up at the desired url:

```
current_url()
```

```
## [1] "https://ingham-equalization.rsgis.msu.edu/"
```

Global Functions

Other actions that apply globally to the entire page:

Function	Task
<code>open_url()</code>	navigate to the indicated url
<code>current_url()</code>	get the current url
<code>get_page_source()</code>	get the page's HTML
<code>back()</code> , <code>forward()</code>	navigate forward or back
<code>reload()</code> , <code>refresh()</code>	reload the current page
<code>take_screenshot()</code>	take screenshot of current image
<code>execute_js_fn()</code> , <code>execute_js_expr()</code>	execute a JavaScript function

2. Navigate to Map Viewer

Now we'll **click the "Start the Viewer" button** to launch the app.

Involved Steps:

- **Select** the button element by its CSS selector (`find_element()`)
- **Scroll to** the button (`elem_scroll_to()`)
- **Click** the button (`elem_click()`)

Selection Functions

Some additional selection functions:

Function	Task
<code>s()</code> , <code>ss()</code>	select HTML elements (without specifying the session)
<code>find_element()</code>	find a single HTML child element (yields a <code>selenium-element</code> object)
<code>find_elements()</code>	find multiple HTML child elements (yields a <code>selenium-elements</code> object)
<code>elem_filter()</code> , <code>elem_find()</code>	extract a subset of HTML elements

Selection Functions

Hold up James, you're telling me that there are **six** different functions ??
Which should I be using?

Thoughts:

- `s()` is equivalent to `session %>% find_element()`
- `ss()` is equivalent to `session %>% find_elements()`
- `s()/find_element()` and `ss()/find_elements()` let you search by
 - CSS Selector (`css`)
 - XPath (`xpath`)
 - element ID (`id`), class (`class_name`), name (`name`)
- `elem_find()` and `elem_filter()` let us search by **conditions**
 - i.e. has certain text, has an attribute with a certain value, is visible, has a certain number of elements
 - We'll see helper functions that will help us out with this in a bit

Element Properties

And functions for obtaining element properties

Function	Task
<code>elem_attr()</code> , <code>elem_attrs()</code> , <code>elem_value()</code>	Get attributes of an element
<code>elem_css_property()</code>	Get a CSS property of an element
<code>elem_name()</code>	Get the tag name of an element
<code>elem_size()</code> , <code>length(selenium- elements)</code>	get the number of elements in a collection
<code>elem_text()</code>	Get the text inside an element
<code>elem_equal()</code>	Test if two elements are equal

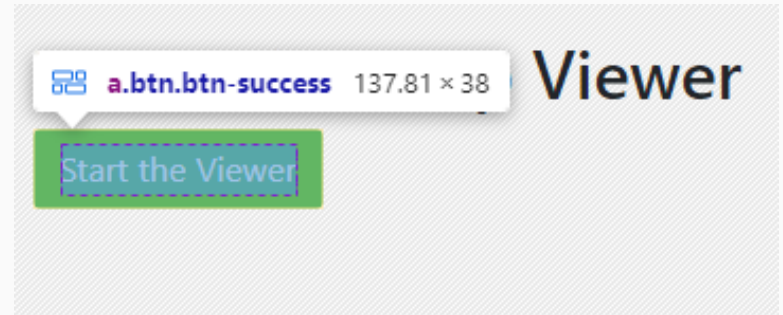
Actions

Function	Task
<code>elem_click()</code> , <code>elem_double_click()</code> , <code>elem_right_click()</code>	Click on an element
<code>elem_hover()</code> , <code>elem_focus</code>	Hover over an element
<code>elem_scroll_to()</code>	Scroll to an element
<code>elem_select()</code>	Select an HTML element
<code>elem_set_value()</code> , <code>elem__send_keys()</code> , <code>elem_clear_value()</code>	Set the value of an input
<code>elem_submit()</code>	Submit an element
<code>keys</code>	list of special keys

2. Navigate to Map Viewer

```
session %>%  
  find_element("a.btn.btn-success")  
  elem_text()
```

```
## [1] "Start the Viewer"
```



```
session %>%  
  find_element("a.btn.btn-success") %>%  
  elem_scroll_to() %>% # move the cursor to the button  
  elem_click() # click the button  
  current_url()
```

```
## [1] "https://ingham-equalization.rsgis.msu.edu/Viewer"
```

3. Search by Address

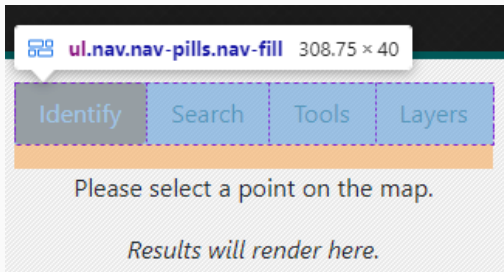
We next want to select + click on the "Search" tab button.

To do this, we'll take advantage of **relative position** selector functions.

Function	Task
<code>elem_parent()</code>	select the element that contains the current element
<code>elem_ancestors()</code>	select every element containing the current element (children, grandchildren, etc.
<code>elem_siblings()</code>	select every element which has the same parent as the current element (e.g. list)
<code>elem_children()</code>	select every element connected to and directly below the current element
<code>elem_descendants()</code>	select every element that is contained by the current element (any type of ancestor)

3. Search by Address

Select + click on the "Search" tab button.

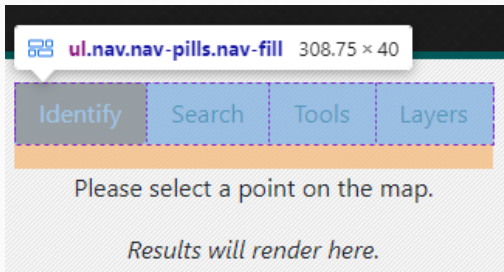


```
session %>%  
  find_element("nav.info-panel") %>% # get full  
  elem_children()
```

```
## { selenider_elements (3) }  
## [1] <ul role="navigation" class="nav nav-pills n  
## [2] <div class=""><div class="hide_container"><b  
## [3] <div class="hide_container" id="layers-tab">
```

3. Search by Address

Select + click on the "Search" tab button.



```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill") %>  
  elem_children()
```

```
## { selenider_elements (4) }  
## [1] <li class="nav-item"><a href="#" class="nav-  
## [2] <li class="nav-item"><a href="#" class="nav-  
## [3] <li class="mobile--hidden nav-item"><a href=  
## [4] <li class="nav-item"><a href="#" class="nav-
```

3. Search by Address

We can also use **condition functions** to help with selection.

Function	Task
<code>has_text(), has_exact_text()</code>	has text that partially or fully matches
<code>has_attr(), attr_contains(), has_value()</code>	does the attribute match a value
<code>has_css_property()</code>	does an element's CSS property match a value (class, id, title, etc.)
<code>has_name()</code>	does an element have a tag name
<code>has_length(), has_size(), has_at_least()</code>	does a collection have a certain number of elements
<code>is_present(), is_in_dom(), is_absent()</code>	does an element exist
<code>is_visible(), is_displayed(), is_hidden(), is_invisible()</code>	is an element visible

3. Search by Address

We can also use **condition functions** to help with selection.

- `find_elements()` to yield a `selenium_element`
- `elem_find()` to extract a specific subset of HTML elements
 - `has_text()` to grab the element with the text "Search"
- `elem_click()` to click the selected button.

```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill") %>%  
  find_elements("li.nav-item") %>%  
  elem_find(has_text("Search")) %>%  
  elem_click()
```

3. Search by Address

Checking to make sure we've activated the "Search" menu tab:

```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill") %>%  
  elem_children()  
  
## { selenider_elements (4) }  
## [1] <li class="nav-item"><a href="#" class="nav-link">Identify</a></li>  
## [2] <li class="nav-item"><a href="#" class="nav-link active">Search</a></li>  
## [3] <li class="mobile--hidden nav-item"><a href="#" class="nav-link">Tools</a></li>  
## [4] <li class="nav-item"><a href="#" class="nav-link">Layers</a></li>
```

See the `nav-link active` class on the second element with the "Search" text?

3. Search by Address

Now **enter text** into the desired fields

```
# Find house number field
session %>%
  find_element("div.service-accord") %>%
  elem_children()
```

```
## { selenider_elements (2) }
## [1] <div class="__title"><h3>Search</h3><button
## [2] <div class="__content"><div class="service-i
```

3. Search by Address

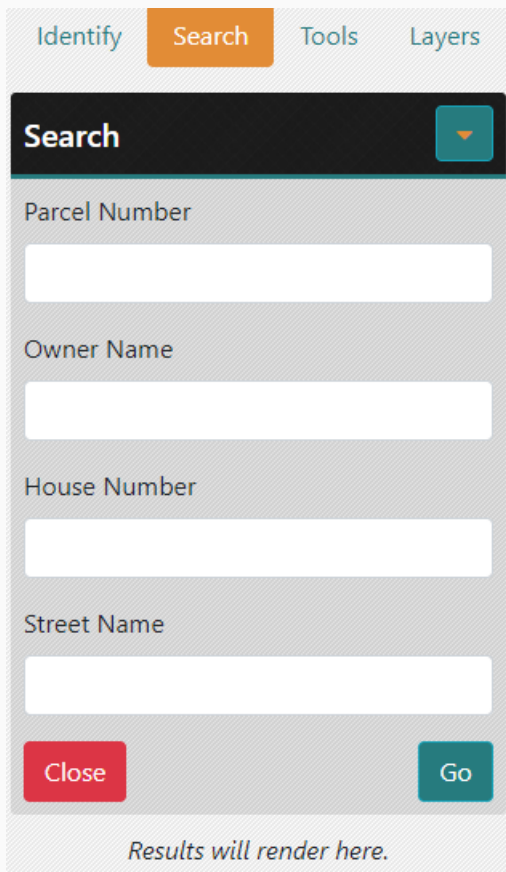
Now **enter text** into the desired fields

```
# Find house number field
session %>%
  find_element("div.__content") %>%
  elem_children()
```

```
## { selenider_elements (5) }
## [1] <div class="service-input form-group"><label
## [2] <div class="service-input form-group"><label
## [3] <div class="service-input form-group"><label
## [4] <div class="service-input form-group"><label
## [5] <div class="tab-controls mr-auto"><button cl
```

3. Search by Address

Now **enter text** into the desired fields and click "Go"



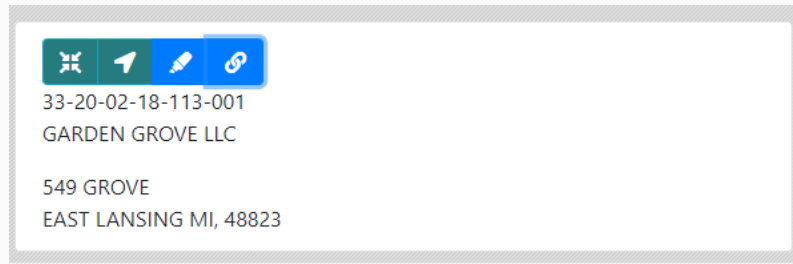
The screenshot shows a search interface with a top navigation bar containing 'Identify', 'Search' (highlighted in orange), 'Tools', and 'Layers'. Below this is a 'Search' panel with a dark header and a dropdown arrow. The panel contains four input fields: 'Parcel Number', 'Owner Name', 'House Number', and 'Street Name'. At the bottom of the panel are two buttons: a red 'Close' button and a teal 'Go' button. Below the panel, a text label reads 'Results will render here.'

```
# set house number
ss("div.service-input.form-group") %>%
  elem_find(has_text("House")) %>%
  find_element("input") %>%
  elem_set_value("549")
```

```
# set street name
ss("div.service-input.form-group") %>%
  elem_find(has_text("Street")) %>%
  find_element("input") %>%
  elem_set_value("Grove")
```

```
# Click button
ss("button.btn.btn-info") %>%
  elem_find(has_text("Go")) %>%
  elem_scroll_to() %>%
  elem_click()
```

4. Retrieve Displayed Info



```
session %>%  
  find_element("div.result-item") %>%  
  read_html() %>%  
  html_text2() %>%  
  str_replace_all("\n\n", "\n") %>%  
  str_split("\n")
```

```
## [[1]]  
## [1] "33-20-02-18-113-001"      "garden grove llc"      "549 grove"  
## [4] "east lansing MI, 48823"
```

5. Follow Detail Link

```
s("a.btn.btn-primary") %>%  
  elem_attr("href") %>%  
  open_url()
```

```
current_url()
```

```
## [1] "https://bsaonline.com/SiteSearch/SiteSearchDetails?SearchFocus=Assessin"
```

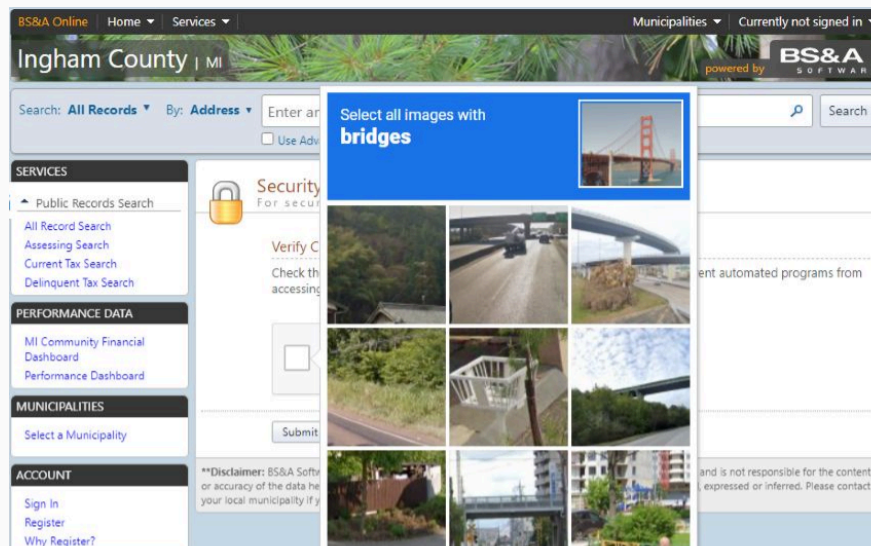
6. Pass Verification Test

Here's where things break down a bit.

We can get our browser to click the "Verify" button

```
s("#GetUserValidationInfo") %>%  
  elem_scroll_to() %>%  
  elem_click()
```

But then we get... the dreaded captcha



6-7. Pass Verification Test + Scrape

While I have yet to figure out a way around one of these image recognition captchas, here's a **semi-automated workaround**:

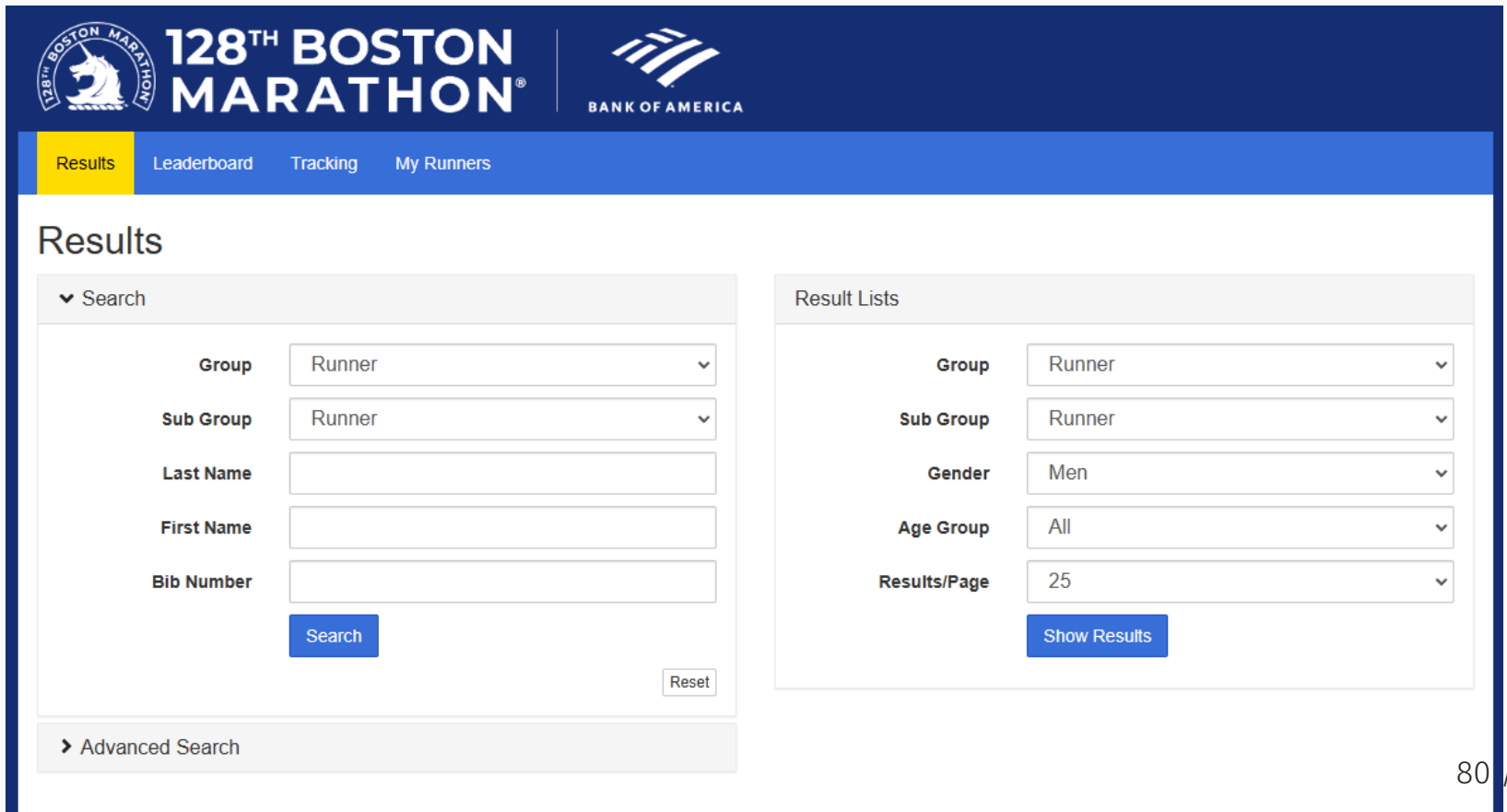
- Wait a set amount of time until an HTML element is present with `elem_expect()` and `is_present()`
 - i.e. element from the results page after the captcha
- Manually complete the captcha (must be viewing the session!)
- Proceed to scrape what's on the page

```
details ← ss("div.record-details-collapsible-box") %>%  
  elem_find(has_text("Building Information")) %>%  
  find_element("div.widthContainer table.detailTable") %>%  
  elem_expect(is_present, timeout = 60) %>% # wait up to 60 seconds to make sure  
  read_html() %>%  
  html_table() %>%  
  as.data.frame()
```

```
details
```

Back to Boston Marathon

Now that we've learned how to use dynamic scraping methods with **selenium**, let's go back to our initial motivation: scraping **2024 Boston Marathon results**



The screenshot displays the official website for the 128th Boston Marathon, sponsored by Bank of America. The page features a dark blue header with the marathon logo and the Bank of America logo. Below the header is a navigation bar with tabs for Results, Leaderboard, Tracking, and My Runners. The 'Results' tab is currently selected, highlighted in yellow. The main content area is titled 'Results' and contains two side-by-side filter panels. The left panel, titled 'Search', includes dropdown menus for 'Group' and 'Sub Group' (both set to 'Runner'), and text input fields for 'Last Name', 'First Name', and 'Bib Number'. It also has a blue 'Search' button and a 'Reset' button. The right panel, titled 'Result Lists', includes dropdown menus for 'Group' (set to 'Runner'), 'Sub Group' (set to 'Runner'), 'Gender' (set to 'Men'), 'Age Group' (set to 'All'), and 'Results/Page' (set to '25'). It features a blue 'Show Results' button. At the bottom left of the 'Results' section, there is a link for 'Advanced Search'.

128TH BOSTON MARATHON[®] | BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

▼ Search

Group: Runner ▼

Sub Group: Runner ▼

Last Name:

First Name:

Bib Number:

Search

Reset

► Advanced Search

Result Lists

Group: Runner ▼

Sub Group: Runner ▼

Gender: Men ▼

Age Group: All ▼

Results/Page: 25 ▼

Show Results

Back to Boston Marathon

Workflow:

- Start a **selenium** session (or use the currently-open one)
- Navigate to the [2024 Boston Marathon results search page](#)
- Select the "Result Lists" form (`form_lists_default`)
- Use the "Age Group" form to select the "18-39" option
- Use the "Results/Page" dropdown to select 100

Back to Boston Marathon

First, navigate to the [2024 Boston Marathon results search page](https://results.baa.org/2024/?pid=start&pidp=start)

```
open_url("https://results.baa.org/2024/?pid=start&pidp=start")
```

And select the form using the `form_lists_default` name:

```
find_element(session, name = "form_lists_default")
```

```
## { selenider_element }  
## <form class="form-horizontal" name="form_lists_default" id="form_lists_default"  
##   \n<fieldset class="fs-event">\n<input type="hidden" name="lang" id="default"  
## </form>
```

Back to Boston Marathon

Within the form, we want to interact with a `select` element. To do this, use `elem_select()` on the desired option

- Here, I used XPath syntax for the "18-39" option.
- SelectorGadget gives *another* XPath that works: `//*[@id="default-lists-age_class"]/option[2]`

```
find_element(session, name = "form_lists_default") :  
  find_element(name = "search[age_class]") %>% # se  
  find_element(xpath = "//*/option[@value = \"1\"]")  
  elem_select()
```

```
<select placeholder class="form-control" name="search[age_class]"  
id="default-lists-age_class">  
  <option value="%">All</option>   
  <option value="1">18-39</option>   
  <option value="2" selected="selected">40-44</option>   
  <option value="9">45-49</option>   
  <option value="3">50-54</option>   
  <option value="10">55-59</option>   
  <option value="4">60-64</option>   
  <option value="11">65-69</option>   
  <option value="8">70-74</option>   
  <option value="12">75-79</option>   
  <option value="13">80+</option>   
</select>
```

Back to Boston Marathon

Next, use the same approach with `elem_select()` to change the results per page option to 100

- This time, using SelectorGadget's CSS selector!

```
find_element(session, name = "form_lists_default") :  
  find_element(name = "num_results") %>% # select t  
  find_element("#default-num_results > option:nth-c  
  elem_select()
```

```
<select class="form-control" name="num_results" id="default-num_results">  
  <option value="25" selected="selected">25</option>  
  <option value="50">50</option>  
  <option value="100">100</option>  
  <option value="500">500</option>  
  <option value="1000">1000</option>  
</select>
```

Back to Boston Marathon

Finally, find and click the button!

```
find_element(session, name = "form_lists_default") :  
  find_element(name = "submit") %>%  
  elem_click()
```

```
<fieldset class="fs-buttons">  
  <div class="form-group field-submit">  
    <div class="col-sm-offset-4 col-sm-8"> == $0  
      <button class="btn btn-primary selectorgadget_selected" type="submit"  
        name="submit" id="default-submit">Show Results</button>  
    </div>  
  </div>  
</fieldset>
```

Back to Boston Marathon

Our page has now been populated with the matching search results!

Retrieve the page's current HTML with `get_page_source()`, and then we can use **rvest** methods to retrieve the text we want.

```
Sys.sleep(5)
res_page1 ← get_page_source() %>%
  html_elements("div.row") %>%
  html_elements("li.list-group-item.row") %>%
  html_text2()
head(res_page1)
```

```
## [1] "Place Overall\nPlace Gender\nPlace Division\nName\nTeam\nTeam\nBIB\nBIB"
## [2] "1\n1\n1\nLemma, Sisay\nTeam\n-\nBIB\n2\nHALF\n01:00:19\nFinish Net\n02:"
## [3] "2\n2\n2\nEsa, Mohamed\nTeam\n-\nBIB\n8\nHALF\n01:02:16\nFinish Net\n02:"
## [4] "3\n3\n3\nChebet, Evans\nTeam\n-\nBIB\n1\nHALF\n01:02:08\nFinish Net\n02:"
## [5] "4\n4\n4\nKorir, John\nTeam\n-\nBIB\n7\nHALF\n01:02:08\nFinish Net\n02:0"
## [6] "5\n5\n5\nKorir, Albert\nTeam\n-\nBIB\n14\nHALF\n01:02:08\nFinish Net\n02:0"
```

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

A:

```
page1_df <- data.frame(string = res_page1[2:length(res_page1)]) %>%  
  separate_wider_delim(cols = string, delim = "\n",  
                        names = c("place_overall", "place_gender", "place_division"),  
                        dplyr::select(-ends_with("dup"))
```

```
head(page1_df)
```

```
## # A tibble: 6 × 9
```

```
##   place_overall place_gender place_division name      team bib   half  finish  
##   <chr>         <chr>         <chr>         <chr>    <chr> <chr> <chr> <chr>  
## 1 1           1           1           Lemma,... -      2    01:0... 02:06:  
## 2 2           2           2           Esa, M... -      8    01:0... 02:06:  
## 3 3           3           3           Chebet... -      1    01:0... 02:07:  
## 4 4           4           4           Korir,... -      7    01:0... 02:07:  
## 5 5           5           5           Korir,... -     14    01:0... 02:07:  
## 6 6           6           6           Mpofo,... -     12    01:0... 02:08:
```

```
## # i 1 more variable: finish_gun <chr>
```

Scraping Dynamic/Interactive Websites

Methods for scraping dynamic or interactive websites opens up a huge range of possibilities.

- Iterative process of
 - Interact with the website and its elements
 - Extract element contents as with static sites
- Interactive workflows are generally more complex, so worth thinking about tradeoff between just brute forcing vs. coding up a scraping routine

Things will also get exponentially easier once we get through our programming lecture, which will allow you to automate repeated scraping tasks

Table of Contents

1. [Prologue](#)
2. [Scraping Static Websites](#)
3. [Interacting with Static Websites](#)
4. [Scraping Dynamic Websites](#)